

1. Declare a function called `getUserInitials` that takes a user's full name (like 'Virat Kohli') and returns their initials in uppercase (e.g., 'VK'). Call this function with your favorite cricketer's name and print the result.

```
function getUserInitials(fullName) {  
    let words = fullName.split(" ");  
  
    let first = words[0][0];  
    let last = words[words.length - 1][0];  
  
    return (first + last).toUpperCase();  
}
```

```
let initials = getUserInitials("Virat Kohli");
```

```
console.log("Initials:", initials);
```

Output:

Initials: VK

2. Create a function `addToCart` that takes a shopping cart array and a product name, adds the product to the cart, and prints the updated cart. Demonstrate how passing the cart array by reference allows changes to persist outside the function.
Hint: In languages like JavaScript, arrays are passed by reference. In C/C++, use pointers for reference behavior.

```
function addToCart(cart, productName) {  
    cart.push(productName);  
  
    console.log("Updated Cart:", cart);  
}
```

```
let cart = ["Mobile", "Headphones"];
```

```
console.log("Before:", cart);
```

```
addToCart(cart, "Smart Watch");
```

```
console.log("After:", cart);
```

Output:

```
Before: [ 'Mobile', 'Headphones' ]
```

```
Updated Cart: [ 'Mobile', 'Headphones', 'Smart Watch' ]
```

```
After: [ 'Mobile', 'Headphones', 'Smart Watch' ]
```

Easy explanation:

The cart array is passed to the function. When we use push(), the **original array is also changed**.

3. Write two functions: increaseFollowersByValue and increaseFollowersByReference. Each should take a followers count and add 1000 to it, but one should use pass-by-value and the other pass-by-reference. Show how the original followers count changes (or doesn't) after calling each function.

JavaScript uses **pass-by-value**, but we can demonstrate reference-like behavior using an object.

Pass-by-Value

```
function increaseFollowersByValue(followers) {  
    followers = followers + 1000;  
    console.log("Inside function:", followers);  
}
```

```
let followers = 5000;
```

```
console.log("Before:", followers);
```

```
increaseFollowersByValue(followers);
```

```
console.log("After:", followers);
```

Output:

Before: 5000

Inside function: 6000

After: 5000

The original value does **not change**.

Pass-by-Reference-like Behavior

```
function increaseFollowersByReference(data) {  
    data.followers = data.followers + 1000;  
}
```

```
let user = {  
    followers: 5000  
};
```

```
console.log("Before:", user.followers);
```

```
increaseFollowersByReference(user);
```

```
console.log("After:", user.followers);
```

Output:

Before: 5000

After: 6000

The object's property changes because the function receives a reference to the same object.

4. Build a function `formatPrice` that takes a price in rupees and returns a string formatted like Flipkart's price tag (e.g., '₹1,599'). Use this function to display prices for three different products.

```
function formatPrice(price) {  
    return "₹" + price.toLocaleString("en-IN");  
}
```

```
console.log(formatPrice(1599));  
console.log(formatPrice(25000));  
console.log(formatPrice(999));
```

Output:

```
₹1,599  
₹25,000  
₹999
```

**5. Refactor an existing function you wrote above to make it reusable for both product names and usernames (for example, a function that capitalizes the first letter of any string).

Constraint: The refactored function should work for any string input, not just a specific use case.**

Instead of making a function only for product names, create a general function that works with **any string**.

```
function capitalizeFirstLetter(text) {  
    return text.charAt(0).toUpperCase() + text.slice(1);  
}
```

```
console.log(capitalizeFirstLetter("mobile"));  
console.log(capitalizeFirstLetter("dhruv"));  
console.log(capitalizeFirstLetter("flipkart"));  
console.log(capitalizeFirstLetter("instagram"));
```

Output:

Mobile

Dhruv

Flipkart

Instagram